
Elevacion



Plan de Pruebas

Versión <1.3>

	Versión: <1.3>
Plan de Pruebas	Fecha: <17/12/2025>
Documento: Plan de Pruebas	

Histórico de revisiones

Fecha	Versión	Descripción	Autor
07/12/2025	1.1	Completado de los puntos 1 al 4 del documento (ambos incluidos)	Francisco Díaz-Hellin
13/12/2025	1.2	Completado de los puntos 5 al 8 del documento (ambos incluidos)	Carlos de la Guía Martínez-Meco
17/12/2025	1.3	Completado de los puntos 9 hasta el 12 del documento (ambos incluidos)	David Gómez Aniorte

	Versión: <1.3>
Plan de Pruebas	Fecha: <17/12/2025>
Documento: Plan de Pruebas	

Índice

1.	Introducción	1
1.1	Objetivos	1
1.2	Ámbito	1
1.3	Referencias	1
2.	Elementos de prueba	1
3.	Características que se deben probar	2
4.	Características que no se van a probar	2
5.	Aproximación	3
5.1	Pruebas funcionales	3
5.2	Pruebas de carga	4
5.3	Pruebas de rendimiento	4
5.4	Pruebas de seguridad	5
6.	Criterios para decidir si pasa/falla un elemento	5
7.	Criterios de suspensión y reanudación	6
7.1	Suspensión	6
7.2	Reanudación	6
8.	Pruebas que se deben entregar	6
9.	Tareas de prueba	6
10.	Necesidades ambientales	7
10.1	Hardware	7
10.2	Software	7
10.3	Seguridad	7
10.4	Herramientas	7
10.5	Publicaciones	7
11.	Responsabilidades.	7
12.	Aprobación	7

	Versión: <1.3>
Plan de Pruebas	Fecha: <17/12/2025>
Documento: Plan de Pruebas	

Plan de Pruebas

1. Introducción

En este documento se trata de identificar el plan de pruebas para nuestro proyecto del concesionario para los tres casos de uso que hemos escogido, los cuales son los casos de uso de la compra de coches, el alquiler de coches y el reseñado de coches. Gracias a este plan de pruebas cualquiera podrá observar que nuestro proyecto cumple con todos los requisitos que se nos han especificado.

1.1 Objetivos

El documento tiene como objetivo principal el comprobar que todas las funcionalidades problemáticas funcionan de acuerdo con las exigencias que nos han brindado, siguiendo tanto los flujos normales como los alternativos.

1.2 Ámbito

El ámbito en el que se van a desarrollar nuestras pruebas es sobre los casos de uso que se nos han brindado y nosotros hemos seleccionado, para los que en todos los casos hay que realizar tres métodos distintos, como son el método "select", que se trata de extraer la lista de coches aplicando unos filtros, el método "details", que trata de devolver al usuario el detalle de la acción que ha realizado y el método "post", que trata de realizar la acción en sí misma.

Las tres acciones que se pueden realizar en nuestro caso, ya que son los casos de uso que hemos elegido es comprar, alquilar y reseñar coches.

1.3 Referencias

Los documentos empleados para llevar a cabo este plan de pruebas serían el documento de casos de uso asociado al grupo B1 (CasosDeUso_grupoB1), en el cual se nos describen todos los flujos tanto el básico y los alternativos, documento con el cual se ha desarrollado todo el proyecto, para concretar más en el desarrollo de este plan de pruebas se ha empleado el documento IEEE Std 829-1998: Standard for Software Test Documentation, el cual es el estándar que se nos ha brindado para definir este documento.

2. Elementos de prueba

Un elemento de prueba es un elemento de software (función, módulo, característica, etc) que debe probarse: versión

Los elementos que se seleccionan para ser probados son los siguientes:

CarsController.cs: Clase encargada de extraer la lista de coches y filtrarlos dependiendo del contexto en el que nos encontramos, devolviéndonos la lista filtrada.

PurchasesController.cs: Clase que guarda la funcionalidad relacionada con la creación de una compra.

RentalsController.cs: Clase que guarda la funcionalidad relacionada con la creación de un alquiler.

ReviewsController.cs: Clase que guarda la funcionalidad relacionada con la creación de una reseña.

DTOs, todos los contenidos en las carpetas CarDTO, PurchaseDTO, RentalDTO y ReviewDTO.

ComponenSelectPurchase: Interfaz utilizada para la selección de coches para la compra.

ComponenentDetailsPurchase: Interfaz utilizada para visualizar el detalle de una compra.

ComponenetPostPurchase: Interfaz utilizada para el formulario de creación de una compra.

ComponenSelectRental: Interfaz utilizada para la selección de coches para el alquiler.

	Versión: <1.3>
Plan de Pruebas	Fecha: <17/12/2025>
Documento: Plan de Pruebas	

ComponenentDetailsRental: Interfaz utilizada para visualizar el detalle de un alquiler.
 ComponenetPostRental: Interfaz utilizada para el formulario de creación de un alquiler.
 ComponenSelectReview: Interfaz utilizada para la selección de coches para una reseña.
 ComponenentDetailsReview: Interfaz utilizada para visualizar el detalle de una reseña.
 ComponenetPostReview: Interfaz utilizada para el formulario de creación de una reseña.

También se probarán todas las clases que guardan los atributos más importantes a la hora de la realización de las distintas acciones de nuestro proyecto como Car.cs, Purchase.cs, Rental.cs, Review.cs, PurchaseItem.cs, RentalItem.cs, ReviewItem.cs.

3. Características que se deben probar

Se deben probar aquellas funcionalidades que afectan a la funcionalidad del concesionario, que en nuestro caso serían las funcionalidades de los tres casos de uso que hemos seleccionado (Comprar coches, Alquilar coches y Reseñar coches) así como el logueo de los usuarios.

Con un riesgo alto (teniendo en cuenta lo importante que sería para un sistema verdadero de un concesionario) tenemos todo el flujo de la compra de un coche, ya que es la característica más común e importante de cualquier concesionario y consta de un alto grado de riesgo ya que un error en las etapas intermedias, como podría ser un cálculo erróneo en el precio total, podría afectar directamente a la empresa económicamente.

Con un riesgo alto, sería el sistema de los alquileres, ya que se debe comprobar que las cantidades que solicita el usuario para alquilar están disponibles, ya que si es capaz de poder alquilar una cantidad mayor de coches a la que verdaderamente hay podría suponer una decisión grave para la empresa ya que ocurriría overbooking al tener disponible la opción de poder alquilar mas coches de los que verdaderamente hay.

Con un nivel de riesgo intermedio tendríamos las reseñas ya que un error en el proceso de las reseñas a pesar de que bloquearía este proceso no afectaría tanto a la empresa ya que un sistema de reseñas no repercute de forma directa sobre la economía de una empresa, al contrario que las compras o los alquileres.

Con un nivel de riesgo alto para nuestro proyecto sería la opción de que el usuario no se pudiera loguear, ya que es una precondition necesaria especificada en el documento que se nos ofreció de casos de uso. A parte de que sería clave en la empresa ya que los usuarios al no poder loguearse no podrían realizar ninguna acción al ser requisito previo para cada caso de uso.

4. Características que no se van a probar

Las características que no se van a probar son aquellas funcionalidades que no se van a probar debido a que en el entorno de las prácticas no son necesarias ya que algunas se simulan.

No se van a probar las pasarelas de pago ya que al estar en un entorno de prácticas no contamos con conexión a bancos reales por lo que está funcionalidad se simula, en un entorno real contaría con un riesgo alto/medio al usuario no poder finalizar ni las compras ni los alquileres.

Tampoco se van a probar los controles a la concurrencia, ya que, al estar la página pensada

	Versión: <1.3>
Plan de Pruebas	Fecha: <17/12/2025>
Documento: Plan de Pruebas	

para un volumen de usuarios ínfimos, no es necesaria la implementación de este control. Pero en caso de un tráfico elevado de usuarios el no tener esta característica puede llevar a que el usuario tenga una mala experiencia de usuario por lo que se le asignaría un nivel de riesgo bajo, ya que no se espera tener un flujo de usuarios demasiado grande.

Por última, otra característica que no se va a tener en cuenta para nuestro proyecto es por ejemplo la devolución de un artículo comprado o alquilado, ya que nuestro proyecto está solo pensado para el proceso de compra, pero no se ha tenido en cuenta la existencia de procesos más allá en la temporalidad. En un sistema real supondría un riesgo medio ya que si el usuario no está satisfecho con su compra, debería ser capaz de tener una plataforma para gestionar sus devoluciones.

5. Aproximación

En el entorno de prácticas que hemos estado desarrollando, principalmente hemos visto 2 tipos de pruebas: pruebas funcionales (que a continuación se explicarán) y pruebas de unidad.

Las pruebas de unidad, es necesario realizarlas para verificar el correcto funcionamiento de los controladores y métodos principales del proyecto .API el cuál es el que soporta el frontend, ubicado en el proyecto .Web. Para cada llevar a cabo todas las pruebas de unidad nos basamos en los conceptos Arrange, Act, Assert. Para la parte "Arrange" en todas las pruebas utilizamos una base de datos reducida (versión Lite de SQL) donde definimos unos datos de prueba con los que posteriormente trabajaremos, a parte de eso definimos en diferentes variables algunos de los resultados que esperamos que se obtengan de la ejecución de las pruebas. Posteriormente, pasamos a la parte "Act" donde propiamente dicho se ejecutan las pruebas de unidad tanto de todos los "select", todos los "post", también todos los "details". Una vez ejecutadas las pruebas, en cada una de ellas se verifica que el resultado que obtuvimos fue el esperado comparándolo con el contenido de las variables que guardan esos resultados esperables.

Estas pruebas están definidas en la carpeta "test" del repositorio en GitHub, en concreto en el proyecto .UT (Unit Testing). Además, han sido llevadas a cabo por todos los componentes del grupo: Francisco Díaz-Hellín Escribano, David Gómez Aniorte y Carlos de la Guía Martínez-Meco

5.1 Pruebas funcionales

En cuanto a las pruebas funcionales, estas están consideradas pruebas de sistema, y su objetivo es verificar que el sistema al completo se ajusta a la especificación de requisitos y restricciones de los casos de uso escogidos al principio de las prácticas.

Estas pruebas son aplicables sobre el frontend (proyecto .Web en el repositorio de GitHub), del proyecto, y para ello es necesario utilizar la herramienta "Selenium" la cual nos permite automatizar estas pruebas sobre la IU. Para ello, se sigue el mismo concepto de "Arrange", "Act" y "Assert", sin embargo, aquí no tenemos una versión reducida de la base de datos SQL, por lo que los cambios se hacen sobre la base de datos original, algo a tener muy en cuenta cuando se estén ejecutando. Para la ejecución de las pruebas tenemos archivos que heredan de la clase "PageObject", que permiten a Selenium encontrar e interactuar con los diferentes elementos de la IU, y archivos cuyo padre es la clase "UC_UIT", los cuales se encargan de dirigir la propia prueba en sí, es decir de ejecutar los tres pasos "Arrange", "Act" y "Assert".

	Versión: <1.3>
Plan de Pruebas	Fecha: <17/12/2025>
Documento: Plan de Pruebas	

Debido a la automatización de estas pruebas, se evita que cualquier tester tenga que interactuar manualmente con todos los elementos de la IU e introducir uno por uno los datos, con tan solo darle al botón de ejecutar, se introducirán todos los elementos de entrada y se comprobarán todas las salidas esperadas en pocos segundos.

Para la realización de estas pruebas hemos participado los 3 componentes del grupo: Francisco Díaz-Hellín Escribano, David Gómez Aniorte y Carlos de la Guía Martínez-Meco

5.2 Pruebas de carga

Para las pruebas de carga se hace uso de la herramienta "JMeter", la cual permite hacer realizar peticiones HTTP hacia una determinada IP, emulando múltiples usuarios. En nuestro caso está página se utilizó para probarla contra otra página web que nada tenía que ver con las prácticas.

Pero si se quisiese probar habría que hacerlo contra la API, estableciendo un total de 200 usuarios realizando peticiones HTTP, con un ramp-up period de 1 segundo (entrada de usuarios de forma escalonada cada 1 segundo) y unas 10 iteraciones de repetición de la prueba.

Hay que tener en cuenta que estas no son cargas de estrés, por lo que el objetivo no es introducir usuarios hasta romper la API, por lo que estos valores para introducir en el software de carga son más que razonables para evaluar su funcionamiento y tiempo de respuesta.

Este software también permite visualizar los resultados de la prueba de carga en forma de árbol o mediante un reporte general, además de establecer distintos límites de tiempo a través de los cuales aceptamos que nos lleguen los mensajes de respuesta afirmativos por parte de la API, de lo contrario la prueba se considera fallida.

Para la realización de estas pruebas no hace falta que las hiciéramos los 3 componentes del grupo, con un componente sería suficiente ya que son pruebas relativamente rápidas de hacer, en este caso se ocuparía David Gómez Aniorte de llevarlas a cabo.

5.3 Pruebas de rendimiento

Las pruebas de rendimiento no se han llevado a cabo en prácticas, pero lo más coherente, para su realización y dado que miden el comportamiento general del sistema en términos de rendimiento; sería llevar a cabo una medición de la media del tiempo de respuesta de la API, es decir, el tiempo medio que tarda la API, en devolvernos un mensaje de respuesta HTTP a cada petición que se realice, teniendo en cuenta que seguramente las peticiones POST conlleven un mayor tiempo de repuesta que las peticiones GET (se tardaría más en añadir una entrada en la correspondiente tabla de la base de datos, que en consultarla y mostrar sus datos).

Para estas pruebas, también se puede usar el software JMeter, ya que tiene un apartado "Duration Assertion" el cual permite un tiempo máximo de respuesta después del cual no se considera válida la prueba que se está realizando.

Y para llevar a cabo las mediciones de la media, se podrían usar los resultados en forma de árbol, la salida general de las pruebas o incluso los logs que se guardan automáticamente en la máquina desde la que se está testeando, una vez la prueba ha finalizado.

Al igual que las anteriores, no hace falta que participen los 3 miembros del grupo, con uno

	Versión: <1.3>
Plan de Pruebas	Fecha: <17/12/2025>
Documento: Plan de Pruebas	

es suficiente, en este caso se encargaría de ellas: Carlos de la Guía Martínez-Meco

5.4 Pruebas de seguridad

Estas pruebas tampoco se van a llevar a cabo en prácticas, pero si las llevásemos a cabo tendrían un objetivo, que sería el verificar si el sistema cumple con la propiedad "security", es decir que es seguro y tienen la capacidad de proteger los datos internos antes ataques externos realizados por terceros. Para ello, se pretende medir e identificar una serie de datos, que se podrán obtener llevando a cabo los siguientes tipos de pruebas de seguridad, como:

- DAST (Dynamic Application Security Testing): son pruebas dinámicas sobre la aplicación en ejecución, que simula ataques para detectar vulnerabilidades, algunos de estos ataques son inyecciones SQL, XSS o CSRF.
- Pruebas de autenticación y autorización: son pruebas que pretenden acceder a sitios de la aplicación a los cuales no están identificados o no tienen los permisos adecuados. Estas pruebas pueden ser llevadas a cabo para acceder a las distintas secciones de "compra", "alquiler" o "reseña", sin pasar antes por la sección de login.
- Pruebas de protección de datos: son pruebas que valida el cifrado de los datos en tránsito o en reposo, por ejemplo, esto se podría aplicar cuando se está llevando a cabo una compra o alquiler cuando el usuario finalmente rellena sus datos y realiza una compra y esta información viaja hasta guardarse en la tabla correspondiente de la base de datos.

Todas estas pruebas deberán de llevarse a cabo en la interfaz de usuario, es decir, contra el proyecto .Web de nuestro repositorio.

Para llevar a cabo se usaran las siguientes herramientas:

- Metasploit
- Acunetix

Y en ellas participan los siguientes miembros: Carlos de la Guía Martínez-Meco y Francisco Díaz-Hellín Escribano.

6. Criterios para decidir si pasa/falla un elemento

En el caso de las pruebas de unidad y funcionales, es muy sencillo y es que el propio Selenium y Visual Studio 2022 informan con un tick verde o una flecha roja, cuando las prueba se ha pasado o no; además deberá de utilizar el documento de Casos de Prueba para introducir las entradas y salidas esperadas en la configuración de las pruebas funcionales, ya esas son las únicas entradas y salidas que han que probar y que se adaptan al caso de uso escogido.

En el caso de las pruebas de carga y rendimiento, también el software JMeter informa de forma visual con un tick verde o una flecha roja, acerca de si la API contestó en el período de tiempo configurado en el programa o no.

Además, para las pruebas de rendimiento, en el caso de que tengamos que hacer una media de los tiempos de respuesta de la API y verificar los resultados, tendremos en cuenta que la prueba se considera exitosa si en la media de todas las peticiones contestó en menos de 750 ms, de lo contrario se considera que el sistema no ofrece el rendimiento esperado y la gran latencia sería percibida por el usuario, algo que sería molesto para su experiencia.

	Versión: <1.3>
Plan de Pruebas	Fecha: <17/12/2025>
Documento: Plan de Pruebas	

7. Criterios de suspensión y reanudación

7.1 Suspensión

- En el caso de las pruebas de unidad, estas deberían de detenerse en el caso de que al menos no estén listos y acabados los métodos "details" de los controladores de cada caso de uso. Sin estos métodos no se puede probar el método post esencial para el funcionamiento del software.
- Las pruebas funcionales, deberán de pausarse en caso de que algún compañero tenga problemas con los logins. Ya que, a pesar de ser una parte muy crítica del código, propensa a errores y difícil de depurar y solucionar sus fallos; sin el correcto funcionamiento de esta parte los testers no podrán ejecutar las pruebas que están desarrollando.
- En el caso de que la API no esté disponible al completo, con todos sus métodos disponibles las pruebas de carga y rendimiento deberán de pausarse hasta que esto ocurra. Esto es necesario, ya que de lo contrario el software de carga no podría realizar todas las peticiones HTTP y tampoco se obtendrían datos estadísticos justos y verídicos.

7.2 Reanudación

- En el caso de las pruebas de unidad, se reanudarán cuando los controladores "details" estén totalmente desarrollados; para este punto habría que volver a ejecutar todas las pruebas de unidad, por si algún cambio realizado en el código ha afectado a lo que ya estaba desarrollado y, en ese caso, fallarían las pruebas unitarias.
- En el caso de las pruebas funcionales, se podrían reanudar cuando la funcionalidad login ya esté disponible en todas las máquinas, para este punto habría que volver a ejecutar las pruebas unitarias y aquellas funcionales que ya estuviesen realizadas en ese punto.
- Por último, para las pruebas de carga y rendimiento, se reanudan cuando la API esté libre de errores y con toda su funcionalidad. Es muy importante, que en este punto se vuelvan a ejecutar las "pruebas unitarias", para verificar su correcto funcionamiento y para asegurarnos que en caso de error en las peticiones HTTP no sea por una incorrecta lógica de negocio en el código, sino por un mal rendimiento en su ejecución.

8. Pruebas que se deben entregar

- Documento de caso de uso "Comprar coches".
- Documento de caso de uso "Alquilar coches".
- Documento de caso de uso "Reseñar coches".
- Plan de pruebas (este propio documento).
- Documento de casos de prueba (unificado en uno solo o uno por cada caso de uso), como bien se prefiera por parte del equipo.

9. Tareas de prueba

Las pruebas principalmente se dividen en tres grupos dependiendo del caso de uso, esto grupos son "Compra de coches", "Alquiler de coches", y "Reseña de coches", para evaluar los distintos grupos de pruebas van a ser tres las personas principales las que se encarguen, Carlos de la Guía Martínez-Meco se encargará de la pruebas del grupo de "Compra de

	Versión: <1.3>
Plan de Pruebas	Fecha: <17/12/2025>
Documento: Plan de Pruebas	

coches”, David Gómez Aniorte las del grupo de “Alquiler de coches” y por último Francisco Díaz Hellín Escribano se encargará de las pruebas del grupo de “Reseña de coches”. Y estas pruebas no tienen dependencias entre sí.

10. Necesidades ambientales

Para el desarrollo de las pruebas se van a tener en cuenta los siguientes factores.

10.1 Hardware

El hardware específico que se emplea en estas pruebas se divide en tres ordenadores cuyos procesadores son Intel core I7 o variantes de la misma potencia y una capacidad de memoria principal de entre 16 y 32 GB.

10.2 Software

El software empleado para la realización de todas las prácticas consiste en el visual studio enterprice junto al software de automatización de pruebas selenium.

10.3 Seguridad

Estas pruebas únicamente serán capaces de ser visibles y ejecutadas por los miembros ya especificados para ejecutar las distintas pruebas de los distintos grupos mencionados ya anteriormente. Los datos sobre los que se ejecutarán las pruebas son locales garantizando así la privacidad de estos y evitando errores por colisiones

10.4 Herramientas

Como se ha mencionado anteriormente, una de las herramientas software que se emplea se selenium para la automatización de estas. Permitiendo así una mayor agilidad de esta y a su vez, que puedan ser escalables.

10.5 Publicaciones

Las publicaciones donde se basa este documento para la realización de pruebas son el documento de casos de uso donde se indican el flujo básico y los distintos flujos alternativos para la creación de lo que se requiera en cada caso de uso. Otros documentos también útiles son los documentos de casos de prueba los cuales han podido facilitar las pruebas.

11. Responsabilidades.

Para cada uno de los grupos mencionados anteriormente con respecto a las tareas de prueba, cada miembro del grupo se encargará de diseñar, preparar y ejecutar su parte, así como la evaluación de estas y los arreglos necesarios de los fallos para que todo sea funcional.

12. Aprobación

El software se unificará y se dará como aprobado una vez que los tres miembros del grupo hayan comprobado que sean correctas, después de eso y que hayan confirmado el resto de los miembros del grupo que las pruebas son correctas, el software completo se unificará y será aprobado por el scrum master de este Sprint (David Gómez Aniorte) (el software que ha implementado el scrum master debe de ser aprobado primero por los miembros del grupo)